

APPFREESN HLD SOPHIX V3 - Illustrated DD-Aligned Update

Version: 2026-06-01

Source PDFs updated:

- APPFREESN-HLD_SOPHIX_V3-120526-124340.pdf
- Finalized DD corpus under 99_rewritten
- Coherence review: 99_analysis/Sophix_V3_DD_Coherence_Review_and_Hardening_20260601.md
- Performance/complexity assessment:
99_analysis/Sophix_V3_Performance_Complexity_Assessment_20260601.md
- MVP runtime simplification baseline:
99_analysis/Sophix_V3_MVP_Runtime_Simplification_Baseline_20260601.md

1. Purpose

This HLD replaces the older SOPHIX V3 high-level design that was written before the DDs were finalized.

The previous replacement was intentionally compact. This version is the fuller HLD: it shows the module landscape, explains why the main design choices exist, and illustrates how modules work together to deliver BSS features.

Use this document as the entry point for:

- architects checking module boundaries;
- developers deciding which DD/API owns a behavior;
- frontend teams designing backoffice, CRM, sales, contractor, and self-care features;
- operations teams sizing and monitoring the platform;
- product owners reviewing end-to-end feature behavior.

Process detail remains in the individual DDs and their developer implementation guides. This HLD explains how those DDs fit together.

Recommended reading order:

1. This HLD.
2. APPFREESN-PROCESS_MODULE_MAP-DD_ALIGNED-20260528.
3. Foundation DDs.
4. The specific module/process DD and DEV_IMPL_GUIDE for implementation.

2. Executive View

SOPHIX V3 is a modular BSS. Each business capability owns its data, APIs, workflows, events, and UI responsibilities. Long-running operations are orchestrated by Camunda. Policy decisions that vary by operator or product are externalized to Drools. Business state is stored in PostgreSQL. Kafka carries facts after they are committed. Redis is a non-authoritative cache only.

The major capability groups are:

Group	Owns	Main DD bundles
Customer and account	Customer master, account master, KYC, contact methods, preferences, notes	09_catalogs/DD_ILM-CFG-01
Catalog and reference data	Service catalog, package catalog, HomePass, tech regions, tax/wallet/discount/equipment configs	09_catalogs, 05_billing/BIL-CFG
Subscription	Subscription master state and subscription operations	02_framework, 03_subscription_lifecycle, 04_subscription_workflows
Billing	Charging, payments, invoices, tax invoices, cycle close, dunning, wallet/top-up, adjustments	05_billing

Group	Owns	Main DD bundles
Fulfillment	Order capture, onboarding, service activation/restriction/termination, HomePass lifecycle fulfillment	06_fulfillment
Work order	Field execution, assignments, finalization, evidence, multi-phase visits	07_work_order
OSR / equipment	Stock chain, equipment instance registry, swaps, RMA, recovery	08_osr_equipment
Ticketing, notification, and workforce	Cases/tickets, ASR ticket flows, customer/staff notifications, internal comms, contractor/staff registry	10_other
Commercial, sales, and franchise	Leads, territories, franchise registry, campaigns, discounts, CVM/retention	10_other, 09_catalogs
Reporting and channels	Reporting mart/API, reporting portal, customer self-care, USSD channel	99_frontend, 10_other/REP-01
Foundations	Auth, DB, Kafka, cache, Camunda, Drools, file storage	01_foundations

2.1 Module Communication And Performance Position

The system has many modules, but it is not designed as all-to-all synchronous communication. Each user journey has one command owner and a small number of supporting reads. The communication model is:

Communication type	Used for	Performance rule
REST/internal API	Commands and current state reads from the owning module.	Keep synchronous chains short. A command should not call many modules in a deep chain.
Kafka events	Facts after commit, reporting feeds, cache invalidation, downstream reactions.	Use outbox/inbox and idempotent consumers. Do not use Kafka as a hidden command bus.
Redis/cache	Reference/catalog reads, session data, read-through acceleration.	Cache is not source of truth. State-changing decisions must revalidate against the owning module.
REP-01 reporting mart	Dashboards, exports, cross-module metrics, management views.	Do not fan out reporting screens to operational modules.
BFF/read aggregator	Customer 360 and dense backoffice screens if latency becomes high.	Aggregates reads only; does not own business state.

Practical implication: a Backoffice Customer 360 screen may show data from ILM, SUB, BIL, TCK, WO, OSR, and NOT, but those calls should be panelized, timeout-protected, and optionally served by a read aggregator. A payment command, activation command, ticket command, or stock command still goes to the owning module only.

The main performance risks are not the number of DDs. They are:

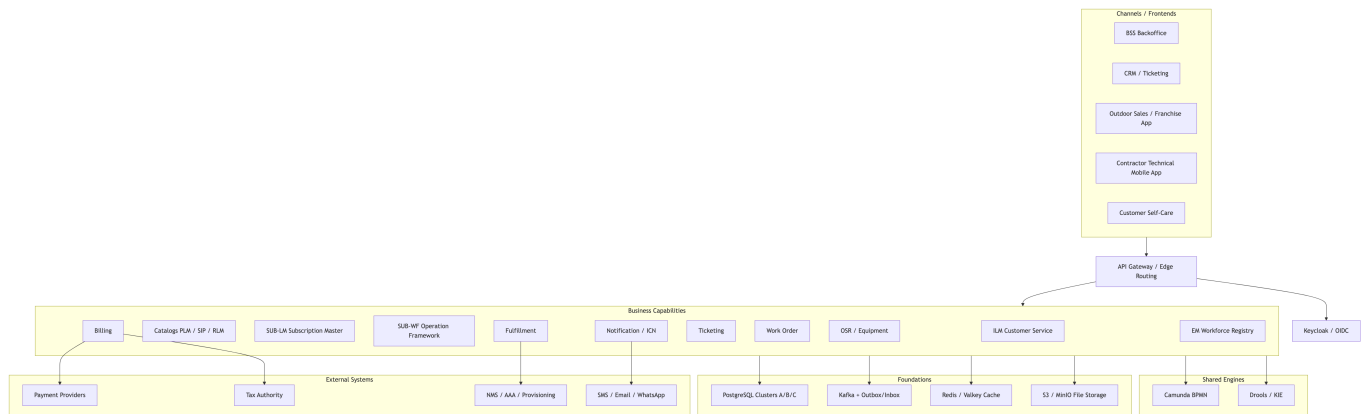
- long synchronous call chains inside commands;
- dashboards querying operational modules instead of REP-01;
- reading another module database directly instead of using APIs/events/read models;
- treating every worker topic as a separate pod;
- using Camunda or Drools for simple deterministic logic.

The mitigation is explicit in the DD set: one write owner per business object, outbox/inbox events, consolidated worker pools for low-volume tasks, REP-01 for analytics, cache only for non-authoritative acceleration, and BFF/read aggregation only for frontend read composition.

3. System Context

Figure 1 shows the top-level runtime structure. Frontend channels call the API layer through the edge/gateway. Business modules call shared foundations and external systems through approved adapters.

Figure 1 - SOPHIX system context



Important reading points:

- The channels are not allowed to bypass module APIs.
- Keycloak protects every user-facing channel and service-to-service integration.
- Business modules own business logic. External systems are accessed through adapters, but adapters do not own BSS policy.
- PostgreSQL is authoritative. Redis accelerates reads but does not own business state.
- Kafka carries committed business facts, not commands pretending to be facts.
- Camunda coordinates long-running workflows. Drools decides policy.

4. Alignment From Old HLD To Final DDs

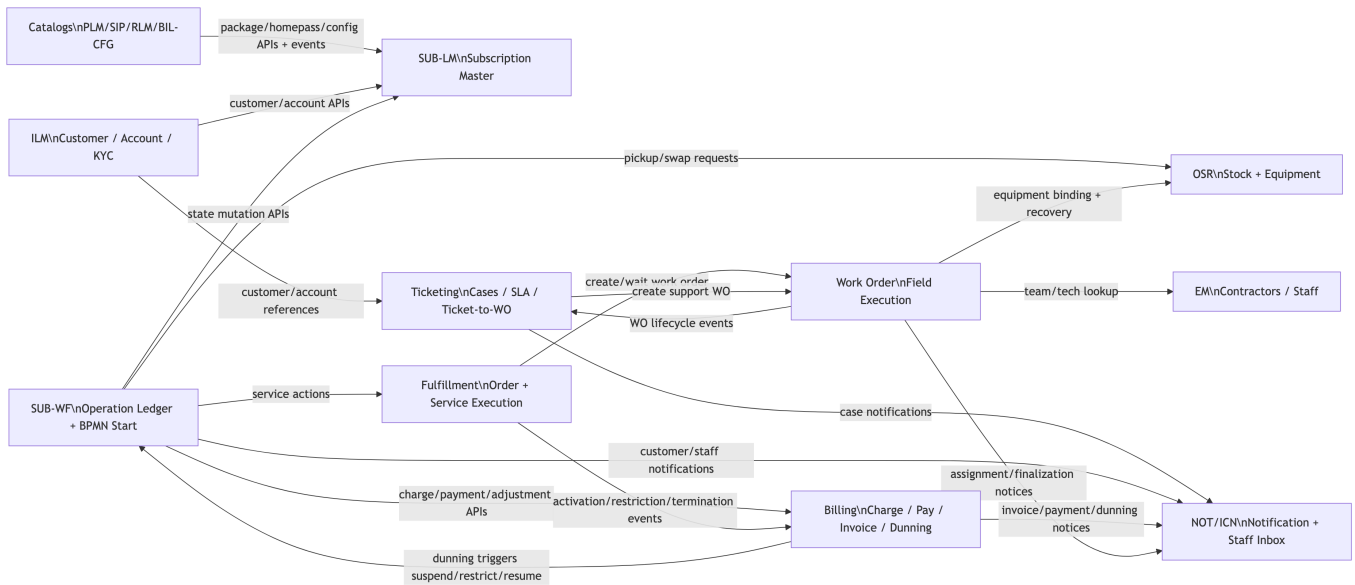
The old HLD used earlier module names and an earlier process inventory. The finalized DD set changed several ownership boundaries.

Old HLD concept	DD-aligned concept	Why it changed
Billing owns customer master	ILM Customer Service owns customer and account	Customer data is used by every domain. Keeping it in billing creates unnecessary financial coupling.
Onboarding as long-lived module	FUL-02 Order Capture owns onboarding workflow	Onboarding is a workflow that ends. Customer, subscription, billing, and fulfillment states continue in their owning modules.
Field-ops module	WO module plus EM workforce registry	Work execution and workforce reference data have different lifecycles and owners.
Inventory module	OSR stock/equipment/RMA capabilities	Stock, equipment instances, swaps, and RMA require explicit lifecycle contracts.
HomePass module	RLM-CFG-01 HomePass Configuration	HomePass is catalog/reference data used by subscription, fulfillment, and work orders.
Single subscription/provisioning process group	SUB-LM plus SUB-WF framework and flow satellites	Subscription master state and operation orchestration must be separated to keep state reliable and workflows flexible.
Workers as deployables	Workers as logical Camunda topics	Operations can consolidate worker apps while preserving per-topic logs, retries, and metrics.

5. Module Collaboration Model

Figure 2 shows the main module dependencies. The arrows are not database joins. They represent REST API calls, Kafka events, or Camunda worker interactions depending on the process.

Figure 2 - Module collaboration model



Key module responsibilities:

Module	Main responsibility	Must not do
ILM Customer Service	Own customer, account, KYC, contacts, preferences, customer search	Must not own billing ledgers or subscription lifecycle state.
PLM/SIP/RLM catalogs	Own product/service/package/HomePass/reference configuration	Must not execute customer-specific operations.
SUB-LM	Own subscription master record and lifecycle fields	Must not orchestrate Camunda workflows.
SUB-WF	Own subscription operation ledger, idempotency, concurrency, Camunda start/correlation	Must not directly mutate subscription rows except through SUB-LM APIs.
BIL	Own money, payments, invoices, tax invoices, dunning, wallet	Must not own customer master.
FUL	Own order capture and service execution state	Must not own subscription master state.
WO	Own work order lifecycle and field execution evidence	Must not own equipment serial authority.
OSR	Own stock, equipment instances, swaps, recovery, RMA	Must not own field scheduling.
TCK	Own tickets/cases, SLA, assignment, ticket timeline, and ticket-to-WO linkage	Must not own customer, WO, billing, subscription, or equipment state.
NOT/ICN	Own notification and internal communication records	Must not decide business policy.
EM	Own contractor/staff registry, franchise registry, RBAC catalog, approval catalog, CVM activities	Must not own work orders, billing ledgers, or ticket lifecycle.
SALES	Own leads, territory assignment, sales activity, conversion attribution	Must not own submitted fulfillment orders.
REP	Own reporting mart, dashboard APIs, exports, reconciliation read models	Must not own operational write state.
ASR	Own service-assurance ticket flow/type rules on top of TCK	Must not create a second ticketing engine.

6. Core Design Choices

6.1 Why ILM Owns Customer

Customer data is required by billing, subscription, fulfillment, work order, notification, and CRM-style screens. If billing owns customer, every non-billing domain becomes financially coupled. ILM Customer Service is therefore the customer/account source of truth.

Implications:

- Billing reads customer/account by ILM API or approved client wrapper.

- SUB-LM stores `account_id` as the operational anchor and `customer_id` as denormalized identity/search context, without cross-module foreign keys.
- Billing invoices, payments, dunning, work orders, and equipment bindings use `account_id` for operational partitioning.
- Customer 360 assembles data from ILM, TCK, SUB-LM, BIL, FUL, WO, OSR, and NOT APIs.

6.2 Why SUB-LM And SUB-WF Are Separate

SUB-LM is the subscription source of truth. SUB-WF is the operation framework that starts and tracks lifecycle/MACD workflows.

This separation prevents workflow complexity from corrupting master data ownership.

SUB-LM owns:

- subscription row;
- subscription status fields;
- cycle fields;
- master fields;
- restrictions catalog and active restrictions;
- low-level subscription events.

SUB-WF owns:

- operation row;
- idempotency key and request hash;
- in-flight operation checks;
- operation history and status;
- Camunda process instance id;
- cancel/interrupt handling;
- operation-level success/failure events.

6.3 Why FUL Owns Order Capture

New customer acquisition touches customer capture, package eligibility, HomePass, KYC, payment, installation, subscription creation, and activation. It is not just a subscription operation.

FUL-02 owns the onboarding/order orchestration because it coordinates fulfillment readiness before activation. Once onboarding finishes:

- ILM owns customer/account;
- SUB-LM owns subscription;
- BIL owns payment/invoice state;
- WO owns field execution history;
- OSR owns equipment;
- FUL owns fulfillment execution records.

6.4 Why WO And OSR Are Separate

Work orders answer: who went where, when, did what, and with what outcome?

OSR answers: where is the equipment, what serial is it, what state is it in, and who is responsible for it?

Keeping them separate avoids mixing field workflow state with inventory/accounting-grade equipment state.

6.7 Why TCK Owns Tickets

Customer Service interactions and notes are not enough for support operations. A ticket is an accountable work item: it has a category, priority, SLA, owner, assignment queue, linked entities, attachments, comments, status, timeline, and resolution. It may also create a Work Order when field intervention is needed.

TCK owns the case lifecycle. WO owns field execution. A technical ticket can call WO-01 POST /api/work-orders, then TCK waits for WO events such as WorkOrderFinalized before resolving or reviewing the ticket.

This keeps the helpdesk/customer-care process separate from the field execution process while linking them through explicit APIs and events.

6.5 Why Camunda And Drools Are Both Used

Camunda is for process orchestration:

- timers;
- human tasks;
- compensation;
- message catches;
- retryable worker tasks;
- long-running multi-module workflows.

Drools is for decisions:

- eligibility;
- operator-specific policy;
- dunning stage decisions;
- tax/fee/discount applicability;
- flow branching decisions.

Drools rules must be side-effect free. They receive facts and return decisions. They do not call HTTP APIs, query databases, publish Kafka, or write files.

6.6 Why Outbox/Inbox Is Required

A domain event is only true after the business state commits. Publishing directly to Kafka inside business code creates a failure window where the database commit and Kafka publish disagree.

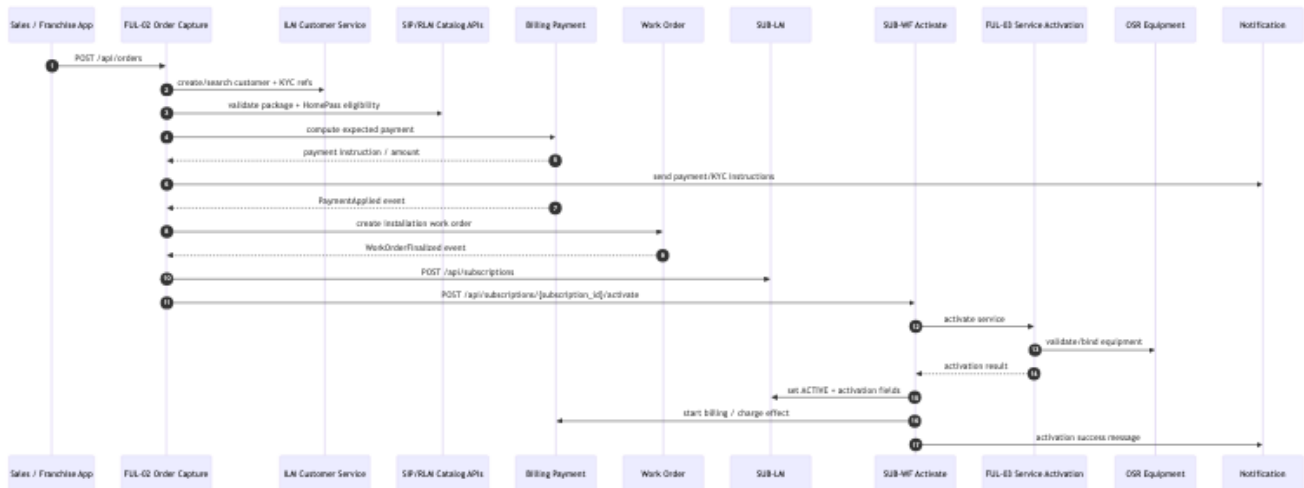
SOPHIX uses:

- producer outbox: business row and event envelope are inserted in the same local database transaction;
- outbox dispatcher: publishes committed outbox rows to Kafka;
- consumer inbox: deduplicates received events before processing;
- idempotent handlers: protect against retries and replay.

7. Feature Delivery Flow - New Customer Sale To Activation

Figure 3 shows how a door-to-door/franchise sale becomes an active subscription.

Figure 3 - New customer sale to activation



Narrative:

1. The sales/franchise app captures prospect, HomePass, package, documents, and contact data.
2. FUL-02 creates the order and starts its Camunda process.
3. ILM creates or updates the customer/account and KYC references.
4. SIP/RLM validate package availability and HomePass eligibility.
5. Billing computes and receives the expected initial payment.
6. FUL-02 creates an installation work order if the flow requires field work.
7. The contractor app completes the work order and uploads evidence.
8. FUL-02 creates the subscription through SUB-LM.
9. FUL-02 calls SUB-WF activation.
10. SUB-WF runs Drools, starts activation BPMN, calls fulfillment/network activation, updates SUB-LM, informs billing, and sends notification.

Main APIs involved:

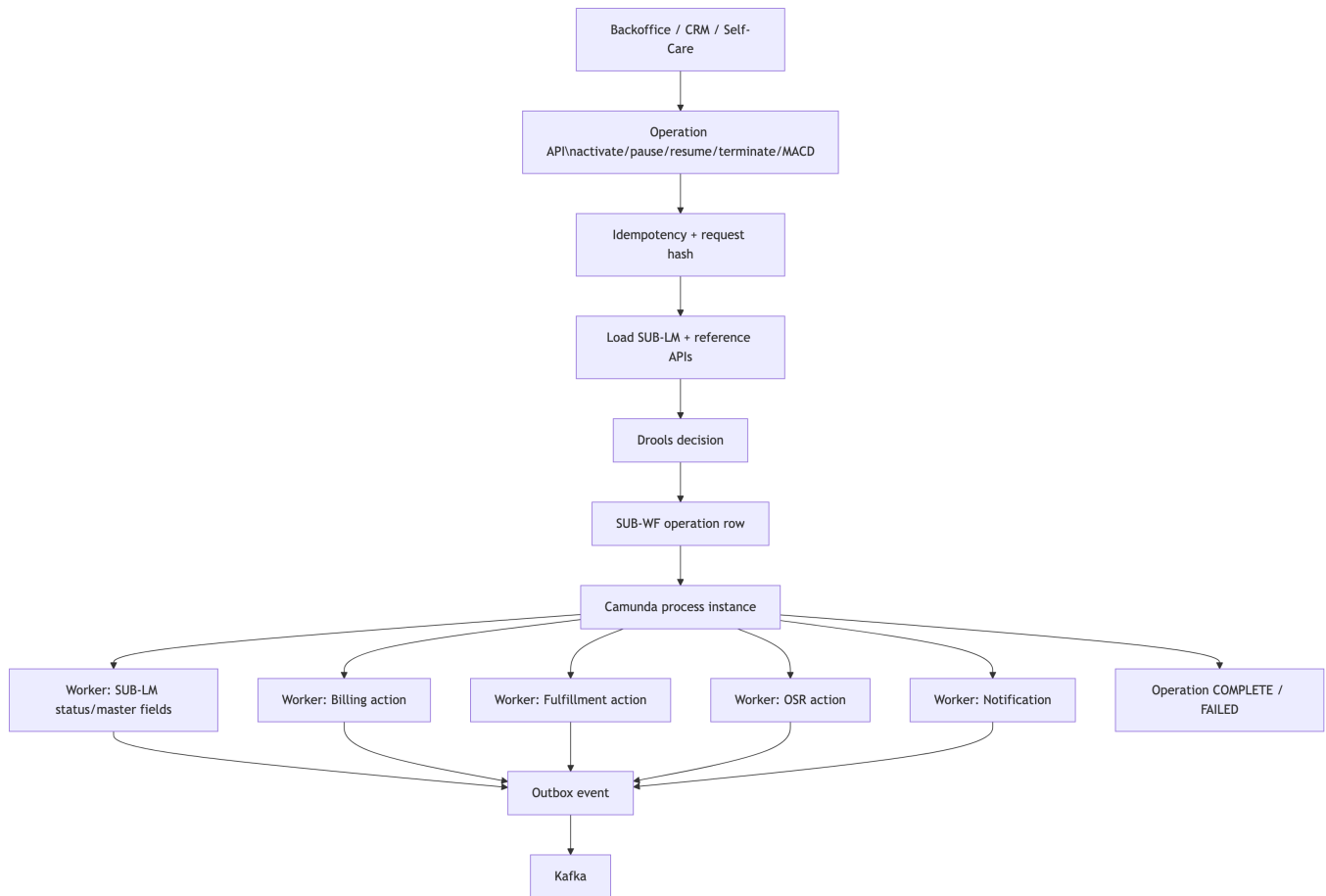
Step	API or event
Order capture	POST /api/orders
Customer create/search	POST /api/customers, GET /api/customers/search
Package/HomePass validation	SIP/RLM APIs or approved read-through client
Payment	POST /api/payments/receive, payment callback event
Ticket	POST /api/tickets, POST /api/tickets/{ticket_id}/work-orders
Work order	POST /api/work-orders, WorkOrderFinalized
Subscription creation	POST /api/subscriptions
Activation	POST /api/subscriptions/{subscription_id}/activate

Design choice: FUL-02 owns the end-to-end onboarding process because it coordinates multiple readiness conditions before activation. SUB-WF owns only the activation operation after a subscription exists.

8. Feature Delivery Flow - Subscription Operations

Figure 4 shows the common runtime pattern for activate, pause, resume, terminate, upgrade, downgrade, restrict, suspend non-payment, relocation, and migration.

Figure 4 - Subscription operation runtime pattern



Common rules:

- The public endpoint belongs to the operation DD, for example POST /api/subscriptions/{subscription_id}/pause.
- The operation API checks auth, request shape, idempotency, and current state.
- Drools decides whether the operation is allowed and which branch/config applies.
- SUB-WF creates the operation row and starts Camunda.
- Camunda external-task topics call SUB-LM, BIL, FUL, OSR, and NOT through approved APIs.
- Final business events are emitted through outbox after state commits.

Idempotency behavior:

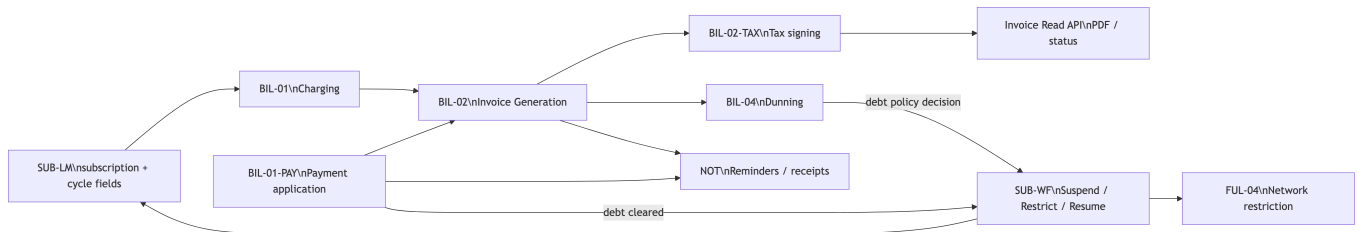
Condition	Behavior
Same key, same request hash	Return existing operation/result.
Same key, different request hash	Return 409 IDEMPOTENCY_KEY_REUSED.
New key	Reserve/insert operation before side effects.

Design choice: the operation row is created before Camunda starts. If Camunda start fails, the operation is marked failed with a controlled reason. This is easier to repair than having an untracked workflow attempt.

9. Feature Delivery Flow - Billing, Payment, Invoice, Dunning

Figure 5 shows how billing features connect to subscription and fulfillment.

Figure 5 - Billing and dunning flow



Billing responsibilities:

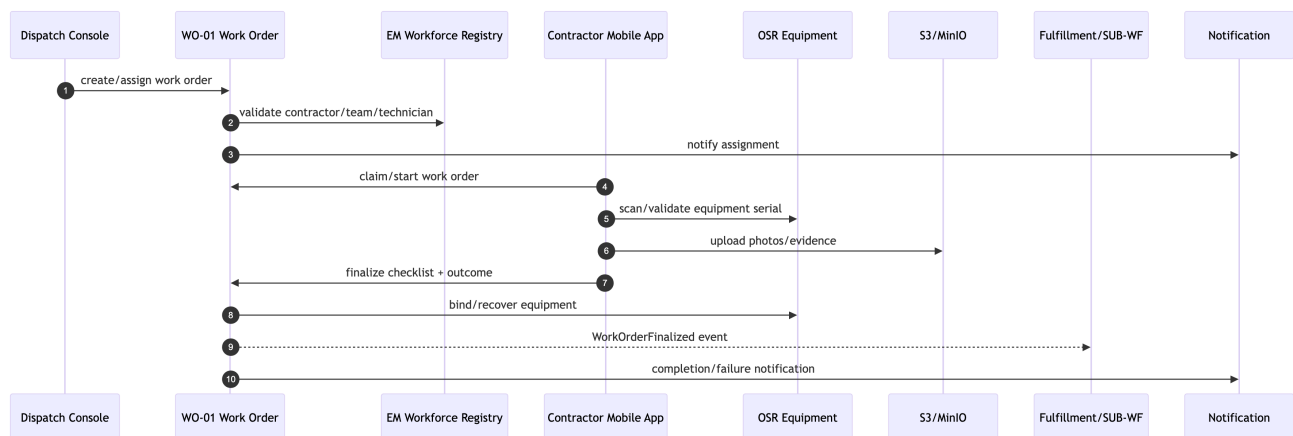
Capability	Owner DD	Notes
Charging	BIL - 01	Converts billable events and catalog rules into charges.
Payment	BIL - 01 - PAY - 01	Receives callbacks, classifies, applies, reverses, emits payment events.
Invoicing	BIL - 02, BIL - 02 - GEN - 01	Generates pro-forma/final invoices and PDFs.
Tax invoice	BIL - 02 - TAX - 01	Integrates with tax authority adapter and handles retry/recovery.
Invoice read/status	BIL - 02 - READ - 01, BIL - 02 - STA - 01	Supports customer care and self-care invoice views.
Adjustments/notes	BIL - 02 - ADJ - 01, BIL - 01 - CN - 01	Handles controlled financial correction flows.
Cycle close	BIL - 03	Scheduled billing period evaluation and close.
Dunning	BIL - 04	Determines debt actions and calls SUB-WF for suspend/restrict/resume.
Wallet/top-up	BIL - 05	Prepaid balance, aliases, top-up channel behavior.

Design choice: dunning does not directly deactivate network service. Dunning decides the commercial action, then calls SUB-WF. SUB-WF coordinates subscription state and FUL restriction/suspension. This keeps billing policy separate from service execution.

10. Feature Delivery Flow - Field Work And Equipment

Figure 6 shows how work orders and OSR equipment collaborate.

Figure 6 - Work order and equipment flow



WO responsibilities:

- create work order;
- assign contractor/team/technician;
- track claim/start/finalization;
- collect notes, photos, attachments, and checklists;
- emit work order lifecycle events.

OSR responsibilities:

- stock balances and movements;
- equipment instance lifecycle;
- equipment serial validation;
- binding and recovery;
- swap/RMA flows.

EM responsibilities:

- contractor registry;
- staff registry;
- team/agent mappings;
- availability references.

Design choice: the contractor mobile app mostly works against WO APIs, but equipment validation and binding go through OSR APIs. The mobile app should not treat scanned serials as valid until OSR confirms them.

11. Event Consistency Pattern

Figure 7 shows how every module publishes and consumes events.

Figure 7 - Outbox and inbox consistency



Producer rules:

- Write the business mutation and outbox row in the same transaction.
- Use a stable event id.
- Use a stable aggregate key so event order is preserved per aggregate.
- Do not wait for Kafka publish before returning the API response unless a DD explicitly says otherwise.

Consumer rules:

- Write to inbox before processing.
- Deduplicate by event id.
- Make the handler idempotent.
- Keep failed inbox rows visible and retryable.
- Do not use a cache update as proof that business state changed.

11A. Performance And Scalability Assessment

The architecture is viable for a medium-budget deployment if implementation teams keep the communication graph controlled. The intended runtime is modular, not chatty.

11A.1 Synchronous Call Limits

Recommended limits for v1:

Use case	Target pattern	Limit
Public command API	Command owner validates and starts local operation.	Prefer 0-3 synchronous module calls before accepting. Push long work to workflow/events.
Backoffice read page	Panelized reads through gateway or BFF.	Timebox each panel; partial page render is acceptable.
Customer self-care/USSD	Small number of direct reads/commands.	Keep interactions short; avoid multi-module screen fan-out.
Reporting dashboard/export	REP-01 only.	No browser-to-operational fan-out for totals.

Use case	Target pattern	Limit
Batch/reconciliation	Scheduled jobs and event consumers.	Avoid running expensive scans during business peaks.

11A.2 Caching Guidance

Use cache for:

- package/catalog/HomePass/reference lookups;
- auth/runtime configuration;
- USSD/session and frontend convenience state;
- short-lived Customer 360 panel acceleration where freshness is labelled;
- cache invalidation based on catalog/config events.

Do not use cache as final truth for:

- balances and payments;
- subscription status transitions;
- work order finalization;
- equipment serial ownership;
- ticket status;
- approval decisions that commit money, stock, or service state.

11A.3 Bottlenecks To Watch

Bottleneck	Mitigation
API gateway saturation from dense backoffice screens	Use pagination, panel lazy loading, and BFF/read aggregation for Customer 360/search queues.
PostgreSQL Cluster C overload	Keep billing/subscription/workflow tables partitioned; tune PgBouncer; move reporting to REP-01.
Kafka consumer lag	Use idempotent consumers, separate high-volume consumer groups, and monitor outbox age.
Camunda incident backlog	Keep only real long-running processes in BPMN; consolidate workers but preserve per-topic metrics.
Redis over-dependence	Treat Redis as acceleration only; keep source APIs healthy.
Reporting pressure on source modules	Feed REP-01 from events and controlled extraction, then serve dashboards from REP-01.

11A.4 Deployment Simplification

The DD/module count does not require one deployment per DD. A medium deployment should group low-volume capabilities:

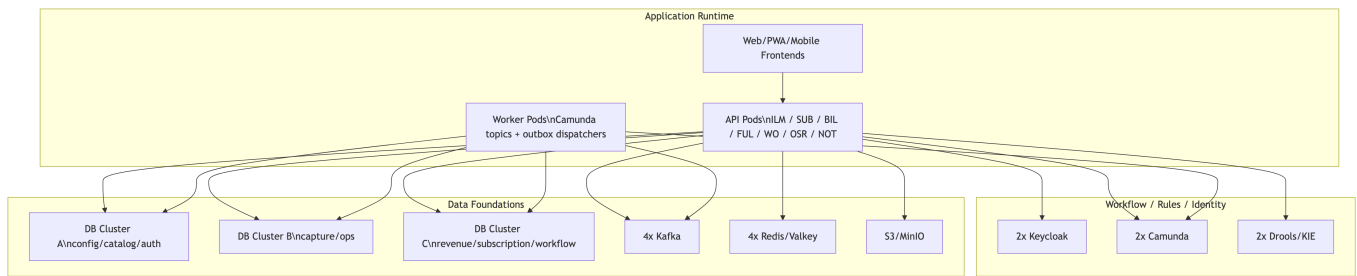
Deployment group	Candidate modules
Customer/catalog API group	ILM, PLM/SIP/RLM read-heavy config APIs where ownership permits.
Subscription API group	SUB-LM and SUB-WF public operation APIs.
Billing API group	Charging, invoice, payment, wallet, dunning APIs.
Field operations API group	WO, EM contractor/staff, field audit APIs.
Equipment API group	OSR stock, equipment instance, RMA/swap, procurement/audit.
Integration worker group	Payment gateway, provisioning, external callbacks/reconciliation.
Reporting/CVM group	REP-01 aggregation APIs, CVM queue generation, exports.

Split a group later when load, release cadence, security isolation, or team ownership requires it.

12. Deployment Topology

Figure 8 shows the medium-budget production topology. It keeps the DB, Kafka, Redis, Keycloak, Camunda, and Drools HA assumptions from the finalized DDs.

Figure 8 - Medium deployment topology



Recommended minimum production shape:

Component	Recommended medium start	Reason
PostgreSQL	6 nodes: Cluster A, B, C each primary/replica	DB is critical; reduce node sizes before reducing topology.
PgBouncer/LB	1 node minimum, 2 preferred	Protect DB connection pools and simplify failover routing.
Kafka	4 nodes, KRaft, RF 3, min ISR 2	Keeps eventing resilient with room for maintenance.
Redis/Valkey	4 nodes, 2 shards x 1 replica	Cache is non-authoritative but should survive one node failure per shard.
Keycloak	2 app nodes plus DB in Cluster A	Login/session availability; JWTs remain locally verifiable.
Camunda	2 app nodes, DB in Cluster C	Workflow state is business critical.
Drools/KIE	2 app nodes	Rules are stateless and can be horizontally scaled.
File storage	2-node MinIO on-prem or S3 on AWS	Stores KYC docs, invoices, work order evidence.
Application APIs	2 pods per critical API	Allows rolling deploy and one-pod failure.
Worker apps	Consolidated worker pools initially	Easier to operate; split high-volume topics later.

Recommended OS baseline:

- Ubuntu Server 22.04 LTS or 24.04 LTS.
- RHEL 9 or Rocky Linux 9 if the ops team standardizes on Red Hat family.
- Same OS and runtime baseline within each role.

Recommended partitioning:

Host type	Partition guidance
App/Camunda/Drools/Keycloak	/ 40 GB, /var/log 20-50 GB, /opt/sophix 20 GB, optional /tmp 10 GB
PostgreSQL	/ 40 GB, /var/lib/postgresql sized for data, optional /var/lib/postgresql-wal 100-300 GB, /var/log 20-50 GB
Kafka	/ 40 GB, /var/lib/kafka 1-2 TB SSD, /var/log 20-50 GB
Redis/Valkey	/ 40 GB, /var/lib/redis 100-200 GB SSD, /var/log 20 GB
MinIO	/ 40 GB, /var/lib/minio sized for retention, /var/log 20 GB

13. PostgreSQL Cluster Placement

SOPHIX keeps three PostgreSQL clusters.

Cluster	Purpose	Hosted databases	Replication
A	Config and catalog	plm, audit, keycloak	Async
B	Customer capture and operations	onboarding, field_ops where retained, operational capture databases	Async

Cluster	Purpose	Hosted databases	Replication
C	Revenue and workflow	billing, billing_cycle_close, billing_dunning, billing_wallet, subscription, fulfillment, camunda	Sync

Cluster C is the critical cluster. Billing, subscription, fulfillment, and workflow state cannot tolerate loss of committed transactions.

Design choice: keep six DB nodes from day one. If budget is limited, start with smaller CPU/RAM/disk dimensions. Do not collapse all SOPHIX databases into one production HA pair because it creates one blast radius for catalog, capture, billing, subscription, fulfillment, and workflow.

14. Frontend Impact Of The Architecture

The HLD directly affects frontend design.

Recommended frontend products:

Frontend product	Main users	Main backend owners called
BSS Backoffice Web	Internal ops/admins	All module APIs behind role-based menus.
CRM / Customer Care / Ticketing	Call center, customer care, supervisors	TCK, ILM, SUB-LM, SUB-WF, BIL, FUL, WO, OSR, NOT.
Outdoor Sales / Franchise App	Door-to-door agents, franchise supervisors	SALES-01, EM-01, FUL-02, ILM, SIP/RLM, BIL/PAY-GW, NOT.
Contractor Technical Mobile App	Contractors, technicians, team leads	WO, OSR, EM-02, FUL, file storage, ICN, TCK for field issues.
Dispatch Console	Dispatchers, field supervisors	WO, EM, OSR, NOT, RLM.
Customer Self-Care	End customers	ILM, SUB-LM, BIL/PAY-GW read/payment/wallet, SUB-WF allowed self-service actions, TCK/ASR, NOT.
USSD Self-Care Channel	End customers on feature phones	Channel adapter over ILM, SUB, BIL/PAY-GW, TCK, NOT.
Platform Admin Console	Security/platform/ops admins	Keycloak, audit, workflow incidents, outbox/inbox, config modules.
Reporting Portal	Management, finance, operations	REP-01 read models, dashboards, exports, reconciliation APIs.

Design choice: multiple frontend products can be one deployable app at first. For example, Backoffice, CRM, Dispatch, Admin, and Reports can share one web app with role-based navigation. Sales, contractor, and customer self-care usually need separate mobile/PWA experiences because their offline, device, and UX constraints differ.

15. AWS Deployment Recommendations

When deployed on AWS, keep the same logical architecture and replace self-managed components with managed services where practical.

SOPHIX component	AWS recommendation
PostgreSQL	Three separate Amazon RDS PostgreSQL deployments/clusters. Use Multi-AZ. Keep Cluster C strongest.
Kafka	Amazon MSK Provisioned for production. MSK Serverless only after throughput/feature limits are accepted.
Redis/Valkey	Amazon ElastiCache for Redis OSS or Valkey.
Object storage	Amazon S3 with SSE-KMS, lifecycle policies, and private access.
Camunda/Drools/API/Workers	EKS or ECS/Fargate in private subnets.
Keycloak	ECS/EKS/EC2 behind ALB, with RDS PostgreSQL.
Secrets	AWS Secrets Manager or SSM Parameter Store.
Observability	CloudWatch metrics/logs and alarms, plus platform dashboards if already standardized.

Network rules:

- App and worker runtimes live in private subnets.
- RDS, MSK, ElastiCache, and S3 access are private.
- Public subnets are used only for approved public load balancers.
- Security groups should be module/role scoped, not broad east-west open rules.
- IAM roles should replace long-lived AWS keys.

16. Observability And Operations

Minimum production dashboards:

Area	Signals
API health	Latency, error rate, 4xx/5xx split, auth failures, idempotency conflicts.
Camunda	Active instances, incidents, failed external tasks, timer backlog, message correlation failures.
Workers	Per-topic success/failure, retry count, age of oldest locked task, dependency failure rate.
Kafka	Under-replicated partitions, offline partitions, broker disk, consumer lag, outbox backlog.
PostgreSQL	Replication lag, locks, pool saturation, slow queries, WAL growth, backup status.
Billing	Payment failures, invoice generation failures, tax signing failures, dunning queue.
Subscription	Stuck operations, conflicting operations, cancel/compensation failures.
Fulfillment/WO/OSR	Activation failures, overdue work orders, equipment binding failures, stock mismatch.
Notification	Provider failure, retry backlog, template errors, delivery failures.

Operational rule: a flow is not production-ready until support can answer:

- where is the process stuck?
- which dependency failed?
- what retry or compensation path exists?
- which event or operation id correlates the incident?

17. DD Traceability

Bundle	Non-guide DD count	Scope
01_foundations	7	Shared auth, DB, Kafka, cache, Camunda, Drools, file storage.
02_framework	3	SUB-LM, SUB-WF framework, worker catalog.
03_subscription_lifecycle	8	Activate, pause, resume, terminate plus WIK flow variants.
04_subscription_workflows	12	Upgrade, downgrade, restrict, suspend non-payment, relocation, migration plus flow variants.
05_billing	13	Charging, payment, notes, invoicing, tax, status, adjustments, cycle close, dunning, wallet, event catalog.
06_fulfillment	14	Order capture framework/steps, service activation, restriction, termination, HomePass lifecycle.
07_work_order	6	Work order module, framework, install, shifting, support, subscription workflow contract.
08_osr_equipment	8	Stock chain, equipment instance registry, swaps/RMA, GPON/HFC/EQP/EQU flows.
09_catalogs	11	Customer service, tech region, service/catalog, tax/wallet/discount/adjustment/equipment/HomePass/package config.

Bundle	Non-guide DD count	Scope
10_other	Current broader bundle	Ticketing/case management, ASR flows, contractor/staff registry, franchise/RBAC/approval/C VM/reporting/field-audit/customer-timeline, internal communications, notification service.
99_frontend	Current frontend set	FE-APP-00 through FE-APP-05 plus USSD channel.

18. Implementation Guidance

Recommended build order:

1. Foundations: auth, DB, Kafka, cache, Camunda, Drools, file storage.
2. ILM Customer Service, catalogs, package management, HomePass config, tech region registry.
3. SUB-LM before SUB-WF.
4. SUB-WF framework and worker catalog before lifecycle flows.
5. Billing payment/charging/invoice basics before dunning/cycle close automation.
6. WO framework and EM registry before FUL installation steps.
7. OSR stock/equipment instance basics before equipment swap/RMA.
8. FUL-02 order capture after ILM, package/HomePass, SUB-LM, payment, and WO basics exist.
9. Subscription activation first, then pause/resume/terminate, then MACD workflows.
10. TCK early enough that Backoffice CRM/ticketing and ticket-to-WO flows do not use notes as a fake ticket system.
11. NOT/ICN early enough that workflows can notify customers and staff without local stubs.
12. REP-01 before management dashboards and exports; do not use operational modules as a reporting backend.
13. PAY-GW, PROV-INT, MED/RAT, OSR procurement/audit, ASR, CVM, and USSD according to the process map and implementation wave.

The old HLD should no longer be used for ownership decisions where it conflicts with this DD-aligned HLD.